APPM 4360 · Spring 2026 · Internal Team Document

Project Implementation Guide

Ideal Fluid Flow in a Polygonal Domain via the Schwarz-Christoffel Transformation

NOT FOR SUBMISSION - for team use only

What We Are Trying to Do

The big picture: we want to show that classical complex analysis (SC conformal mappings) gives us a clean, exact way to solve a fluid flow problem inside an ugly, irregular shape - specifically the Boulder city boundary polygon.

The three-line summary of the math:

- Map Boulder polygon Ω conformally to the upper half-plane $\mathbb{H}$ using the Schwarz-Christoffel transform.
- Solve uniform flow in $\mathbb{H}$ - trivially: complex potential $W(\zeta) = U\zeta$.
- Pull the solution back to Ω via the inverse map - streamlines appear automatically.

The deliverables the instructor actually sees:

- Abstract (due March 13) - one polished page, third person.
- Presentation (April 29) - 15 min + 2 min Q&A, slides with math + figures.
- Written report (due April 27) - full paper, max ~12 pages of content.

Team Roles

Member	Responsibilities
Othello (You) *Code Lead*	• Obtain & simplify Boulder polygon shapefile • Implement SC parameter solver (Python) • Implement inverse map & streamline grid • Produce all publication-quality figures • Write Appendix (code listing)
[Author 2] *Math Lead*	• Write up SC theory derivation in full detail • Derive complex potential for uniform flow • Verify interior angle computations • Write Introduction and Project Body math sections
[Author 3] *Writing Lead*	• Abstract & Conclusions drafting • Bibliography, citations, proofreading

	• Slide deck layout and narrative • Coordinate submission deadlines

NOTE Roles are suggestions - overlap is fine. But Othello owns the codebase and all figures.

Guide for Math Lead ([Author 2])

Your job is to make the mathematics in the report complete, rigorous, and readable. You do not need to write any code - but you do need to understand what the code is doing well enough to explain it in mathematical language.

1. Write the SC Theory Section

Start from [AF03] Chapter 5. Your task is to re-derive the SC formula in your own words - do not just copy the textbook. The logical chain should be:

- State the Riemann Mapping Theorem: any simply connected domain (not all of $\mathbb{C}$) can be mapped conformally to the unit disk. This is the theoretical guarantee that our map exists.
- Explain what conformal mappings preserve: angles locally, harmonic functions, and therefore solutions to Laplace's equation. This is why mapping works for fluid flow.
- Introduce the SC formula: $f(\zeta) = A + C\int_0^{\zeta} \prod_k (t - \zeta_k)^{\alpha_k - 1}\, dt$. Explain what each term means - the exponents $\alpha_k - 1$ encode the interior angles, and the ζ_k are pre-images of the vertices on the real axis.
- State (and prove if space allows) that this formula produces corners of angle $\alpha_k \cdot \pi$ at each vertex - this is the key geometric result.
- Discuss the Möbius normalisation: the SC map has 3 real degrees of freedom (a Möbius group action on $\mathbb{H}$), so three pre-vertices must be fixed. The code fixes $\zeta_0 = -1$, $\zeta_1 = 0$, $\zeta_{n-1} = 1$; the remaining $n - 3$ pre-vertices are the unknowns. Critically, these free pre-vertices must satisfy $0 < \zeta_2 < \zeta_3 < \cdots < \zeta_{n-2} < 1$. The solver enforces this by the softmax parameterisation: unconstrained parameters $p \in \mathbb{R}^{n-3}$ are mapped to strictly monotone pre-vertices via cumsum(softmax([$p_0,\ldots,p_{n-4}$, 0])), giving a smooth, everywhere-differentiable residual that Levenberg-Marquardt can handle cleanly.

NOTE Target length for this block: about 2-3 pages in the report. Every step should be something a fellow APPM student can follow without looking anything up.

2. Derive the Complex Potential for Uniform Flow

This section connects SC mapping to fluid dynamics. The logical chain:

- Define ideal (irrotational, incompressible) flow: velocity field $\mathbf{u} = \nabla\phi$ where $\nabla^2\phi = 0$ (Laplace's equation).
- Introduce the stream function ψ: it satisfies $\nabla^2\psi = 0$ and is harmonic conjugate to φ. Together they form the complex potential $W = \phi + i\psi$, which is analytic.
- In the upper half-plane $\mathbb{H}$, uniform horizontal flow of speed U simply has $W(\zeta) = U\zeta$. Streamlines are $\mathrm{Im}(\zeta) = \text{const}$ - horizontal lines. Trivial.
- Explain the key step: because conformal maps preserve harmonicity, $W(f^{-1}(z))$ is the complex potential in the Boulder domain Ω automatically. No extra solving needed.
- State the no-penetration boundary condition: streamlines are tangent to the boundary. This

holds automatically because the boundary maps to the real axis of $\mathbb{H}$, which is a streamline ($\psi = 0$) of the uniform flow.

3. Verify the Interior Angle Formula

Review the angle computation in src/angles.py (function interior_angles_pi) and verify it against the formula. For each vertex k, the interior angle in units of π is:

$$\alpha_k = 1 - \frac{\theta_k^{\text{ext}}}{\pi}$$

Check that the sum $\sum_k \alpha_k = n - 2$ for the simplified Boulder polygon (function verify_angle_sum in src/angles.py does this automatically and logs PASSED/FAILED). Confirm this is exactly n − 2; any deviation signals a non-simple polygon or CW orientation. Write this up as a brief paragraph in the report and include the numerical value.

$$\sum_{k=1}^{n} \alpha_k = n - 2$$

4. Explain the Figures in Mathematical Language

Once the figures are produced, your job is to write the in-text discussion. There are now eleven figures. For Figures 2-4 (uniform flow), discuss:

- Why streamlines compress near convex corners (conformal map stretches the metric there - the Jacobian $|f'(\zeta)|$ is large).
- Why streamlines are tangent to the boundary everywhere (no-penetration, by construction).

$W(\zeta) = U\zeta + \frac{Ua^2}{\zeta-\zeta_0} + \frac{Ua^2}{\zeta-\bar{\zeta}_0}$e orthogonality between streamlines and equipotential lines represents (Cauchy-Riemann equations, conformal grid).For Figure 5 (terrain-informed flow): explain how per-vertex source/sink terms W_terrain = $\sum_k \frac{q_k}{2\pi}[\log(\zeta - s_k) + \log(\zeta - \bar{s}_k)]$ modify the streamlines, why the image term $\log(\zeta - \bar{s}_k)$ is necessary (method of images: $\mathrm{Im}(W) = 0$ on $\mathbb{R}$), and why RBF thin-plate spline + 5-fold CV R^2 is reported instead of in-sample R^2.For Figure 7 (urban obstacle): state the circle theorem formula

$$W(\zeta) = U\zeta + \frac{Ua^2}{\zeta - \zeta_0} + \frac{Ua^2}{\zeta - \bar{\zeta}_0}$$

$$W_{\text{terrain}}(\zeta) = U\zeta + \sum_k \frac{q_k}{2\pi}[\log(\zeta - s_k) + \log(\zeta - \bar{s}_k)]$$

> **TIP** Read the figure first, then write the explanation. Do not write generic math and hope it matches the figure - look at what actually happened and explain that specifically.

Guide for Writing Lead ([Author 3])

Your job is to make the project coherent and readable as a whole - from the first sentence of the Introduction to the last line of the Bibliography. You are responsible for the narrative, not the derivations.

1. Write the Introduction (≤ 2 pages)

The Introduction should answer four questions in order:

- Why do we care about fluid flow in irregular domains? (Engineering, urban flooding,

ventilation, etc. - pick one and make it concrete.)

- Why is this mathematically hard in general? (Laplace's equation in an irregular geometry has no closed-form solution - you need a trick.)
- What is the trick? (Conformal mappings transform the domain into something simple where we can solve exactly, then map back. This idea goes back to the 19th century - Schwarz and Christoffel independently developed the transformation in the 1860s-1870s.)
- What does this paper do specifically? (We apply the SC transform to the Boulder city boundary polygon, solve uniform flow, and visualize the result. One short paragraph that previews the whole paper.)

> **NOTE** The Introduction should be readable by any math-literate person, not just someone who has taken complex variables. Avoid symbols in the first paragraph. Lead with the physical intuition, then bring in the math.

2. Write the Conclusions & Future Directions (≤ 2 pages)

Conclusions: summarize what was actually accomplished - what we computed, what the figures show, and what the mathematical takeaway is. Do not introduce new ideas here. About half a page.

Future Directions: this is where you can be creative. Some ideas to develop into full paragraphs:

- Terrain-corrected flow [IMPLEMENTED]: elevation data from USGS 3DEP (EPQS API, queried at polygon vertices + boundary interpolation points + interior grid) is fitted with a 2-D thin-plate-spline RBF surface (scipy.interpolate.RBFInterpolator). Generalisation accuracy is reported as 5-fold cross-validation R² (in-sample R² is always 1.0 for interpolators and is meaningless). Per-vertex source/sink singularities in $\mathbb{H}$ are placed at s_k = ζ_k + 0.25i with strength q_k proportional to the projection of ∇e onto the free-stream direction, capped at Q_scale = 0.35. Image terms $+\log(\zeta - \bar{s}_k)$ enforce $\psi = 0$ on $\mathbb{R}$ exactly.

$$W_{\text{terrain}}(\zeta) = U\zeta + \sum_k \frac{q_k}{2\pi}[\log(\zeta - s_k) + \log(\zeta - \bar{s}_k)]$$

$W(\zeta) = U\zeta + \frac{Ua^2}{\zeta-\zeta_0} + \frac{Ua^2}{\zeta-\bar{\zeta}_0}$bstacle / doubly-connected domain [IMPLEMENTED]: the downtown commercial core is obtained via OSMnx (fallback: synthetic convex hull). Its boundary is mapped into $\mathbb{H}$ via the inverse SC map, and the minimum enclosing circle is found with Welzl's O(n) randomised algorithm (exact MEC, not centroid approximation). The Milne-Thomson circle theorem + method-of-images gives $\psi = 0$ on $\mathbb{R}$ exactly and no-penetration on the obstacle circle up to $O((a/\mathrm{Im}(\zeta_0))^2)$ error.

$W(\zeta) = U\zeta + \frac{Ua^2}{\zeta-\zeta_0} + \frac{Ua^2}{\zeta-\bar{\zeta}_0}\psi = 0O((a/\mathrm{Im}(\zeta_0))^2)$

- Road-vortex flow [IMPLEMENTED]: major road intersections downloaded from OSM via osmnx (primary/secondary/tertiary roads), mapped to $\mathbb{H}$ via SC inverse map, and modelled as point vortices with image charges. Circulation strength proportional to node degree (number of roads meeting). Figures 9-10.

$$W(\zeta) = U\zeta + \frac{Ua^2}{\zeta - \zeta_0} + \frac{Ua^2}{\zeta - \bar{\zeta}_0}$$

- Other city shapes: apply the same pipeline to a different city (Manhattan is a near-rectangle; Los Angeles is wild). Makes for a nice comparison figure.
- Viscous flow: ideal flow ignores viscosity. A natural extension is to use the Navier-Stokes equations and compare the streamlines to the ideal-flow result.

3. Own the Bibliography

You are responsible for making sure every claim in the report that needs a citation has one, and every reference in the bibliography was actually used. Minimum requirements:

- [AF03] Ablowitz & Fokas - cite in the Introduction (SC theory) and Project Body (formula derivation).
- [DT09] Driscoll & Trefethen - cite when describing the SC parameter problem and the numerical method.
- [BO99] Bender & Orszag - cite for complex potential / ideal flow background.
- US Census Bureau, TIGER/Line Shapefiles - cite as the data source for the Boulder boundary shapefile (Place boundaries, Colorado, 2025, tl_2025_08_place).
  - USGS National Map, 3DEP Elevation Point Query Service (EPQS) - cite when describing the terrain elevation data acquisition pipeline.
  - Boeing, G. (2017). OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks. Computers, Environment and Urban Systems, 65, 126-139 - cite for the urban polygon acquisition via OSMnx.

AVOID Do not cite sources you have not read. If a fact comes from a textbook, open the textbook, find the page, and cite the specific section. The instructor will check.

4. Proofread the Entire Report

Before submission, do one full pass of the complete document looking for:

- Every figure mentioned in the text with 'Figure N shows...' - if a figure is not discussed in the text, it must be removed per the style guide.
- Consistent notation: make sure Math Lead and Othello are using the same symbols (e.g., always ζ for the half-plane variable, always z for the physical domain).
- Third-person voice throughout (required by the abstract; good practice for the whole paper).
- 11pt font, no handwriting, standard bibliography style - per the project style guide.

5. Build the Slide Deck Narrative

The presentation is 15 minutes. Suggested structure (not the math lead's job to decide - this is yours):

- Slide 1: Title + team. 30 seconds.
- Slides 2-3: Motivation. What is ideal flow, why does domain shape matter, what is Boulder. Keep it visual - show a photo of Boulder and the polygon outline side by side.
- Slides 4-6: The math idea. One slide per step of the pipeline (warp → solve → unwarp). Use simple diagrams, not dense formulas.
- Slides 7-9: Results. The streamline figure is the hero. Spend real time here explaining what the audience is looking at.
- Slides 10-13: Extended models. One slide each for terrain-corrected flow, urban obstacle flow, and road-vortex flow — show streamlines side-by-side with the uniform baseline. Final slide: four-panel summary figure.

TIP Do not put the SC formula on a slide and read it out loud. Instead, draw a before/after diagram: Boulder on the left, rectangle on the right, arrow labeled 'SC map' in the middle. That communicates more in 10 seconds than two minutes of formula-reading.

The Math Pipeline (What the Code Must Implement)

Step A - Schwarz-Christoffel Formula

The SC map $f: \mathbb{H} \to \Omega$ is:

$$f(\zeta) = A + C \int_0^{\zeta} \prod_k (t - \zeta_k)^{\alpha_k - 1}\, dt$$

$$f(\zeta) = A + C \int_0^{\zeta} \prod_k (t - \zeta_k)^{\alpha_k - 1}\, dt$$

where $\zeta_k \in \mathbb{R}$ are pre-images of the polygon vertices on the real axis, $\alpha_k \cdot \pi$ are the interior angles, and *A, C* are complex constants (scale + translation). The ζ_k are unknown and must be solved for - this is called the SC parameter problem.

Step B - The SC Parameter Problem

The pre-vertex locations ζ_k must satisfy nonlinear cross-ratio constraints to ensure the image polygon has the correct side-length ratios. This system has no closed form for n ≥ 4 and must be solved numerically.

> **NOTE** The SC parameter solver is implemented from scratch using scipy: side lengths are computed via Gauss-Legendre quadrature on the real axis, and the pre-vertex positions are found by Levenberg-Marquardt least squares. This custom implementation converges well for the Boulder polygon (n = 14, residual cost 3.64×10^{-2}). An external library (polygon-sc / pySC) is an alternative if the custom solver proves unstable for other polygons.

Step C - Flow in the Upper Half-Plane

In $\mathbb{H}$, uniform flow of speed *U* in the x-direction has the simple complex potential:

$$W(\zeta) = U\zeta \qquad \Rightarrow \qquad \phi = U\,\mathrm{Re}(\zeta), \quad \psi = U\,\mathrm{Im}(\zeta)$$

$$W(\zeta) = U\zeta \qquad \Rightarrow \qquad \phi = U\,\mathrm{Re}(\zeta), \quad \psi = U\,\mathrm{Im}(\zeta)$$

Streamlines are $\psi =$ const, i.e. horizontal lines in $\mathbb{H}$. Beautifully simple.

Step D - Pull Back to Boulder Domain

For each grid point z in Omega, evaluate W at the pre-image zeta via the SC forward map: sample a dense grid of zeta values in H, map forward z = f(zeta) via Gauss-Legendre integration, evaluate W(zeta) = U*zeta at each point, then interpolate (griddata) to the regular output grid. Streamlines are level curves of psi = Im(W). No inverse solver is needed - the forward map is the approach used throughout (src/flow.py: compute_flow_grid).$z \in \Omega \zeta = f^{-1}(z) W(\zeta) = U\zeta \psi(z) = \mathrm{Im}(W(f^{-1}(z)))$

Coding Guide (Othello's Roadmap)

Step 1 - Get the Boulder Polygon

Download Boulder city limits from the US Census TIGER/Line shapefiles (Place boundaries, Colorado, 2025). Free download, no account needed.

- URL: https://www.census.gov/geographies/mapping-files/time-series/geo/tiger-line-file.html
- Use FIPS code 08013 (Boulder County) or search 'Boulder city, CO'.

- Load into Python with geopandas:

```
import geopandas as gpd
gdf = gpd.read_file('tl_2025_08_place.shp')
boulder = gdf[gdf['NAME'] == 'Boulder'].geometry.values[0]
```

Reproject to UTM Zone 13N (meters) so distances are Euclidean - critical for correct angle computations:

```
gdf = gdf.to_crs(epsg=26913)
```

> **TIP** Always work in a projected CRS (meters), never in WGS84 degrees. Angle computations in lat/lon are distorted.

Step 2 - Simplify the Polygon

The raw city boundary has 1935 vertices. SC solvers become numerically unstable for n > ~20 (pre-vertices cluster — the 'crowding problem'). Simplify to the target range [min_vertices, max_vertices] using Douglas-Peucker with an adaptive binary-search tolerance:

```
# src/polygon.py  simplify_polygon()  —  adaptive binary search
lo, hi = 1.0, tolerance * 20.0   # bracket the tolerance
for _ in range(50):
    simplified = polygon.simplify(tol, preserve_topology=True)
    n = len(simplified.exterior.coords) - 1
    if min_vertices <= n <= max_vertices: break
    if n > max_vertices: lo = tol
    else:                hi = tol
    tol = (lo + hi) / 2.0
# Boulder converges to ~240 m tolerance, 11 vertices after angle smoothing
coords = list(simplified.exterior.coords)[:-1]   # drop closing repeat
import numpy as np
pts = np.array(coords)   # shape (n, 2)

# Visualize to sanity-check
import matplotlib.pyplot as plt
x, y = simplified.exterior.xy
plt.plot(x, y); plt.axis('equal'); plt.title('Simplified Boulder'); plt.show()
```

> **WARN** Tolerance=500 (meters) is a starting guess. Adjust until the shape still looks roughly like Boulder but vertex count is under 16.

Convert to complex numbers and normalize to O(1) scale for numerical stability:

```
z_poly = pts[:,0] + 1j*pts[:,1]
z_poly -= z_poly.mean()
z_poly /= np.abs(z_poly).max()
```

Step 2.5 - Angle Smoothing (prevent SC crowding)
Even after Douglas-Peucker, some vertices may have interior angles outside [alpha_min * pi, alpha_max * pi]. These cause SC crowding: the corresponding pre-vertices collapse to machine-epsilon distance from each other, making all quadrature integrals inaccurate. Remove them iteratively before solving:
# src/polygon.py smooth_extreme_angles()
from src.angles import interior_angles_pi

alpha_min, alpha_max = 0.15, 1.85 # in units of pi

```
changed = True
while changed and len(z_poly) > min_vertices:
    changed = False
    alphas = interior_angles_pi(z_poly)
    severity = np.maximum(alpha_min - alphas, alphas - alpha_max)
    worst = np.argmax(severity)
    if severity[worst] > 0:
        z_poly = np.delete(z_poly, worst)   # drop the offending vertex
        changed = True
# Boulder: 1935 raw -> ~14 after D-P -> 11 after angle smoothing
```

This is the single most important pre-processing step. Without it the SC solver will appear to converge but the forward map will be badly distorted near the removed vertices. The actual thresholds used are [0.35*pi, 1.75*pi] (see main.py --min-vertices / --max-vertices flags).

Step 3 - Compute Interior Angles

The SC exponents $\alpha_k - 1$ come from the interior angles of the polygon. Compute them from the vertex coordinates:

```
def interior_angles_pi(pts_complex):
    n = len(pts_complex)
    alphas = []
    for i in range(n):
        v_in  = pts_complex[i] - pts_complex[i-1]
        v_out = pts_complex[(i+1) % n] - pts_complex[i]
        # Exterior turn angle
        turn = np.angle(v_out / v_in)
        # Interior angle = π - turn  (in units of π)
        alphas.append(1.0 - turn / np.pi)
    return np.array(alphas)   # should sum to n - 2
```

> **NOTE** For a convex polygon all $\alpha_k \in (0,1)$. For a non-convex (re-entrant) vertex, $\alpha_k > 1$. The sum must equal n − 2 exactly - use this as a sanity check.

Step 4 - Solve the SC Parameter Problem

This is the crux of the numerical work. The solver is implemented from scratch in src/sc_solver.py. Side lengths are computed via vectorised Gauss-Legendre quadrature (n_pts = 500 nodes per segment; semi-infinite tails truncated at R = 500, giving ~0.2% truncation error). The free pre-vertices are parameterised via the softmax transform to guarantee strict monotonicity without bounds constraints, then optimised with Levenberg-Marquardt (scipy.optimize.least_squares, method='lm'). No external SC library is required.

```
# No external SC library required - all implemented in src/sc_solver.py
```

If the package is unavailable, implement a minimal version using scipy:

```
from scipy.integrate import quad
from scipy.optimize import least_squares

def sc_integrand(t, zk, alphas):
    val = 1.0 + 0j
    for z, a in zip(zk, alphas):
        val *= (t - z) ** (a - 1.0)
```

```
    return val

def sc_side_length(zk_a, zk_b, zk_all, alphas):
    # Integrate |f'(t)| from zk_a to zk_b along real axis
    result, _ = quad(lambda t: abs(sc_integrand(t, zk_all, alphas)),
                     zk_a, zk_b, limit=200)
    return result

def parameter_residuals(free_params, fixed_zk, alphas, target_ratios):
    # Reconstruct full zk:
    # Möbius normalisation fixes zk[0]=-1, zk[1]=0, zk[n-1]=1.
    # Remaining n-3 free pre-vertices are in (-1, 0) ∪ (0, 1),
    # reparameterised via softmax so they stay strictly ordered
    # without explicit box constraints (smooth, differentiable).
    # Softmax parameterisation: p → strictly monotone zk_free in (0,1)    w =
np.append(np.exp(p - p.max()), 1.0)   # numerically stable    w /= w.sum()
zk_free = np.cumsum(w[:len(p)])            # strictly increasing in (0,1)    zk
= np.empty(n); [fix ζ₀=-1,ζ₁=0,ζₙ₋₁=1 from fixed_vals]    for k, idx in
enumerate(free_idx): zk[idx] = zk_free[k]
    sides = [sc_side_length(zk[i], zk[i+1], zk, alphas)
             for i in range(len(zk)-1)]
    ratios = np.array(sides[:-1]) / sides[-1]
    return ratios - target_ratios
```

AVOID Fixing the Möbius normalisation is essential. The SC map has 3 complex degrees of freedom (a Möbius transform on $\mathbb{H}$), so three pre-vertices must be fixed. The implementation uses $\zeta_0 = -1$, $\zeta_1 = 0$, $\zeta_{n-1} = 1$ (three finite real points). The remaining $n - 3$ pre-vertices lie in the interval (0, 1) and are the free unknowns. Without this normalisation the system is underdetermined and the solver will fail.

Step 5 - Evaluate the Forward Map $f(\zeta)$

Once the parameters ζ_k are solved, evaluate the SC integral numerically for any $\zeta \in \mathbb{H}$:

```
from scipy.integrate import quad_vec

def sc_map(zeta_arr, zk, alphas, A, C):
    results = []
    for zeta in zeta_arr:
        def integrand(t):
            return np.prod([(t - z)**(a-1) for z,a in zip(zk, alphas)])
        val, _ = quad(integrand, 0, zeta.real)  # NOTE: actual implementation
uses an L-shaped path in ℍ (see sc_solver.py)
        results.append(A + C * val)
    return np.array(results)
```

Determining A and C: once zk is solved, A and C are found by an overdetermined least-squares fit. For each of the n polygon vertices compute the SC integral from a fixed reference point zeta_ref = 0.5j (safely in $\mathbb{H}$, away from all branch points): I_k = integral_{zeta_ref}^{zk[k] + 0.05j} integrand dt. Then solve the (2n x 2)-real linear system A + C * I_k = z_poly[k] for k=0..n-1 in the least-squares sense. This globally minimises vertex error rather than enforcing only two calibration points. See src/sc_solver.py _solve_AC().

TIP For points in the interior of $\mathbb{H}$, integrate along a contour from 0 to $\mathrm{Re}(\zeta)$, then vertically to ζ.

Use scipy.integrate.quad with complex output or integrate real/imag parts separately.

Step 6 - Forward Map and Streamline Grid

Sample a dense grid of ζ values in $\mathbb{H}$, map each point forward via z = f(ζ), evaluate W(ζ) = Uζ, then interpolate to a regular z-space grid. For exact streamlines, horizontal lines Im(ζ) = y_0 are mapped forward as parametric curves: $f(\zeta)$

```
import numpy as np
from src.flow import compute_curves_forward

# Forward map: horizontal lines Im(zeta) = y0 in H -> streamlines in Omega
y_levels = np.logspace(-1.4, 0.8, 28)   # 28 streamline levels
t_vals = np.linspace(-20.0, 20.0, 300)  # parametric parameter

stream_curves = []
for y0 in y_levels:
    zeta_line = t_vals + 1j * y0
    z_line = sc_map(zeta_line, params)   # forward map: H -> Omega
    # Clip to polygon interior, split at boundary crossings
    stream_curves.append(clip_to_polygon(z_line, norm_polygon))

# Equivalently, use the helper from src/flow.py:
stream_curves, equip_curves = compute_curves_forward(params, norm_polygon)
```

WARN This loop is slow (n_grid × n_grid root solves). During development use --grid 30 or --grid 40. The default production grid is 80×80 (6,400 interior root solves). Use --grid 120 only for final report figures if quality is insufficient at 80.

Step 7 - Final Figure

This is the main visual result of the paper. Make it publication-quality:

```
fig, ax = plt.subplots(figsize=(9, 9), dpi=150)

# Boulder boundary
x_b, y_b = simplified.exterior.xy
ax.fill(x_b, y_b, color='#F0F4FA', zorder=0)
ax.plot(x_b, y_b, color='#1A3A6B', lw=2.0, zorder=2)

# Streamlines
```

```
cs = ax.contour(XX, YY, Psi, levels=25,
                colors='#2563A8', linewidths=0.9, zorder=3)

# Equipotential lines (optional, nice to have)
# Phi = Re(f_inv(z)) - compute similarly

ax.set_aspect('equal')
ax.axis('off')
ax.set_title('Ideal Flow Streamlines - Boulder City Polygon', fontsize=14)
plt.tight_layout()
plt.savefig('streamlines_boulder.pdf', bbox_inches='tight')
```

TIP Save as PDF (vector) for the report, and PNG (dpi=300) for slides. Never use default matplotlib resolution for a submitted figure.

Timeline

Dates	Who	Task
Mar 8-13	All	Get instructor approval; finalize & submit abstract
Apr 11-18	Othello	Download shapefile, simplify polygon, get angles, sanity-check visualization
Mar 13-21	Math Lead	Study sc_solver.py and sc_solver_dc.py. Write the full SC theory section: SC formula derivation, interior angles, parameter problem (softmax parameterisation), method of images for terrain, circle theorem for urban obstacle.
Mar 22-Apr 5	Othello	Implement SC parameter solver; forward map; inverse map on test polygon (square first!)
Apr 11-18	Othello	Run full pipeline on Boulder polygon; produce final figures
Apr 5-11	Writing	Read through the code and all generated figures. Draft Introduction (motivation + problem setup) and Background (ideal flow, conformal maps). Agree on full report outline with team.
Apr 18-25	All	Othello writes Methods + Results. Math Lead completes theory sections and slide equations. Writing assembles full report draft and writes Conclusion. Share draft for cross-review.
Apr 25-29	All	Finalize and proofread report. Complete 15-min slide deck. Two full rehearsals. Present April 29. Submit final written report.

NOTE Always test the SC implementation on a known polygon first (square, L-shape) before running on Boulder. You know the answer for a square: flow should look like uniform horizontal lines.

What the Final Results Should Look Like

When everything works, you should be able to produce eleven figures across four physical models:

Figure 1 - Simplified Boulder Polygon

Side-by-side: original high-resolution city boundary vs. the simplified polygon used for computation. Demonstrates that the simplification preserves the essential shape.

Figure 2 - Streamlines in Boulder Domain

The key result figure. Streamlines (level curves of ψ) should:

- Enter from one side of the domain and exit the other.
- Compress and curve near 'pinch points' in the boundary (sharp indentations).
- Never cross each other (they are level curves of a harmonic function).
- Be tangent to the boundary everywhere (no-penetration condition holds automatically by construction).

> **TIP** If streamlines cross or shoot off to infinity, the inverse map solver didn't converge for those points. Check convergence flags and increase the grid of initial guesses.

Figure 3 - Equipotential Lines ($\phi = \text{const}$) [optional but recommended]

Figure 4 - Combined Streamlines & Equipotentials

Overlay of streamlines (blue) and equipotentials (orange). The two families form the conformal grid - the image of a rectangular grid in $\mathbb{H}$ pulled back through f. Orthogonality everywhere is the visual proof that the mapping is conformal.

Figure 5 - Terrain-Informed Flow

Streamlines under the terrain-corrected potential. Implementation pipeline in src/terrain.py: (1) query USGS 3DEP EPQS API at vertices + 3 interpolation points per edge + 25 interior grid points (total ~69 points); (2) fill API failures with a linear west-high east-low Boulder model; (3) fit a 2-D thin-plate-spline RBF to all samples, validated with 5-fold cross-validation (CV R^2 logged at runtime); (4) compute per-vertex elevation gradients via 100m centred finite differences on the RBF surface; (5) project each gradient onto the flow direction (east) to get signed source strength q_k = Q_scale * (grad . e_flow) / max|grad|; (6) place one source (q>0) or sink (q<0) per vertex at s_k = zk[k] + 0.25j in $\mathbb{H}$ with its image below $\mathbb{R}$. Q_scale=0.35 caps the maximum source at 35% of free-stream speed. Red triangles mark the highest vertices (~1770m, west); blue triangles the lowest (~1570m, east).$W(\zeta) = U\zeta + \sum_k \frac{q_k}{2\pi}[\log(\zeta - s_k) + \log(\zeta - \bar{s}_k)]$

$$W_{\text{terrain}}(\zeta) = U\zeta + \sum_k \frac{q_k}{2\pi}[\log(\zeta - s_k) + \log(\zeta - \bar{s}_k)]$$

Figure 6 - Uniform vs. Terrain-Corrected (side-by-side)

Direct comparison at identical contour levels. The terrain correction bends streamlines toward lower elevation; the shift is largest in the western portion where the elevation gradient is steepest.

Figure 7 - Urban Core as Interior Obstacle

Doubly-connected flow: the downtown commercial core is treated as an impenetrable obstacle. The inner polygon boundary is mapped to $\mathbb{H}$ via the inverse SC map; its minimum enclosing circle (found by Welzl's algorithm, centre ζ_0, radius a) defines the obstacle. Streamlines curve around the purple filled obstacle and show a clear stagnation region. The no-penetration error on the circle is $O((a/\mathrm{Im}(\zeta_0))^2)$, logged as 'Separation Im(ζ_0)/a' at runtime.

Figure 8 - Three-Way Comparison

All three models (uniform / terrain-corrected / urban obstacle) at the same contour levels, demonstrating how the conformal mapping cleanly decouples domain geometry from the physical model.

Figure 9 - Road-Vortex Flow

Point vortices placed at the 11 highest-degree OSM road intersections (primary/secondary/tertiary roads). Intersection pre-images in $\mathbb{H}$ found via SC inverse map. Vortex strength proportional to node degree; sign positive (CCW) north of the polygon centroid, negative (CW) south. Streamlines show local eddies at each intersection, consistent with Boulder's prevailing westerly-flow shear from the Flatirons.

Figure 10 - Uniform vs. Road-Vortex Comparison

Direct side-by-side comparison of uniform flow and road-vortex flow at identical contour levels. The road-vortex correction introduces organised local circulation absent from the uniform baseline, particularly along the Broadway and 28th St corridors.

$$\phi = \mathrm{Re}(W(f^{-1}(z)))$$

Figure 11 - Four-Way Comparison (all models)

Four-way side-by-side: uniform flow / terrain-corrected / urban obstacle / road-vortex at the same contour levels. Each panel shows the progressive physical enrichment of the SC framework, from ideal parallel flow to a doubly-connected, terrain- and circulation-informed model. The four potentials are evaluated on the same forward-mapped grid so panel-to-panel differences are due entirely to the physical corrections.

Common Pitfalls

AVOID Working in WGS84 (lat/lon): interior angles will be wrong. Always reproject to UTM first.

AVOID Forgetting to fix 3 pre-vertices (Möbius normalization): the SC parameter system is underdetermined. Fix $\zeta_0 = -1,\ \zeta_1 = 0,\ \zeta_{n-1} = 1$ before solving.

WARN Too many polygon vertices: SC solver diverges for n > ~20. Simplify until n ≤ 16.

WARN Testing directly on Boulder without validating on a simple shape first: if your square doesn't work, Boulder won't work. Always test on a known case.

NOTE The inverse map (root-finding) will occasionally fail to converge for points near the boundary or near singular pre-vertices. Mask those points out in the figure - a few NaN pixels are fine.

TIP Keep a Jupyter notebook as your working scratchpad and a clean .py script as the 'official' code for the appendix. Don't submit a messy notebook.

Key References for the Paper

[AF03] Ablowitz & Fokas, Complex Variables, Cambridge, 2003. → Main textbook; Ch. 5 for SC theory.

[DT09] Driscoll & Trefethen, Schwarz-Christoffel Mapping, Cambridge, 2009. → Must cite; definitive numerical SC reference; cite for the parameter problem and numerical method.

[BO99] Bender & Orszag, Advanced Mathematical Methods, Springer, 1999. → Cite for complex potential / ideal flow context.

US Census Bureau, TIGER/Line Shapefiles, 2025. → Cite as data source for Boulder boundary.

[BG17] Boeing, G. (2017). OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks. Computers, Environment and Urban Systems, 65, 126-139. → Cite for OSM road network and urban polygon acquisition.

NOTE You need ≥ 3 non-internet/non-web references. [AF03], [DT09], and [BO99] cover this requirement. The Census data source is web-only and does not count toward the 3.